# Tell me full implementation what the maths comes here and what new formulas will introduce and also what will be the mathematical equations

Here is the full mathematical design for AgentBudget's loop controller. The key idea is to combine **classical information gain**—reduction in uncertainty about the task—with **agent progress signals**, repetition penalties, and a budget-aware value function. Classical information gain is defined as $IG(Y|X) = H(Y) - H(Y \mid X)$, the reduction in uncertainty about $Y$ after observing $X$. [1] [2]

### Task state

At step $t$, define the agent state:

$$s_t = (g,\ M_t,\ K_t,\ B_t,\ H_t,\ O_t)$$

| Symbol | Meaning |
|---|---|
| $g$ | Task goal and success criteria |
| $M_t$ | Completed milestones |
| $K_t$ | Verified facts / artifacts |
| $B_t$ | Remaining budget |
| $H_t$ | Action history |
| $O_t$ | Tool observations |

An action is:

$$a_t = (\tau_t, x_t)$$

where $\tau_t$ is the tool name and $x_t$ contains its arguments.

The environment returns an observation:

$$o_t = \text{Tool}(\tau_t, x_t)$$

and the state transitions:

$$s_{t+1} = T(s_t, a_t, o_t)$$

### Classical information gain

For a task with a target variable $Y$—for example, “which file contains the bug?” or “which retrieved document answers the question?”—define entropy:

$$H(Y) = -\sum_{y \in \mathcal{Y}} p(y) \log_2 p(y)$$

with the convention $0 \log 0 = 0$. Entropy measures uncertainty in bits. [1] [3]

After observing action $a_t$ and output $o_t$, conditional entropy is:

$$
H(Y \mid o_t)
$$

-\sum_{y \in \mathcal{Y}}
p(y \mid o_t)\log_2 p(y \mid o_t)

$$

Then true information gain is:

$$
IG_t
$$

#### H(Y)

H(Y \mid o_t)

$$

Equivalently, this is mutual information:

$$IG_t = I(Y; o_t)$$

A high $IG_t$ means the observation materially reduced uncertainty about the task. [2] [1]

### Practical estimate

In most agent tasks, you do not know the true posterior $p(y)$. Estimate it using a lightweight scorer, retrieval ranker, or model-based belief update:

$$
\widehat{IG}_t
$$

H(\hat{p}_t)

H(\hat{p}_{t+1})

$$

where:

# $$
# \hat{p}_t(y)

P(Y=y \mid s_t)

$$

and:

# $$
# \hat{p}_{t+1}(y)

P(Y=y \mid s_t, a_t, o_t)

$$

For coding, $Y$ may be the failing test or root-cause hypothesis. For RAG, $Y$ may be the correct answer or supporting document.

### Agent progress gain

Classical information gain alone is not enough: an agent may learn something but still make no progress. Define an **Agent Progress Gain**:

**$$**
**APG_t =**
**\alpha_1 \Delta S_t**
**+**
**\alpha_2 \Delta M_t**
**+**
**\alpha_3 \widehat{IG}_t**
**+**
**\alpha_4 \Delta P_t**

\alpha_5 R_t

\alpha_6 C_t

$$

### State-change term

$$\Delta S_t = \mathbb{1}\left[h(s_{t+1}) \neq h(s_t)\right]$$

where $h(\cdot)$ is a canonical state hash.

For a coding task:

$$s_t = (\text{diff}_t,\ \text{test_status}_t,\ \text{open_failures}_t,\ \text{files}_t)$$

A new failing test can increase progress because it provides a new constraint.

### Milestone term

Let $M_t \subseteq \mathcal{M}$ be completed milestones.

$$\Delta M_t = |M_{t+1}| - |M_t|$$

Example milestones:

# $$
# \mathcal{M}

{
locate bug,,
implement fix,,
tests pass
}

$$

### Evidence novelty

Measure whether the observation is new relative to recent observations:

# $$
# N_t

1 -
\max_{j \in W_t}
\operatorname{sim}(o_t, o_j)

$$

where $W_t$ is a sliding window of previous observations.

For exact duplicates:

# $$
# \operatorname{sim}(o_t,o_j)

\mathbb{1}[h(o_t)=h(o_j)]

$$

For text, use cosine similarity:

# $$
# \operatorname{sim}(o_t,o_j)

\frac{e_t^\top e_j}
{|e_t|_2|e_j|_2}

$$

where $e_t = \mathrm{embed}(o_t)$.

Then:

$$\Delta P_t = N_t$$

### Repetition penalty

Create an action fingerprint:

$$f_t = h(\tau_t, \mathrm{canonical}(x_t))$$

Let:

$$R_t = \mathbb{1}[f_t \in F_{t-w:t-1}] + \lambda_{\text{near}} \mathbb{1}\left[\max_{j \in W_t} \mathrm{sim}(x_t, x_j) > \theta_x\right]$$

where $F_{t-w:t-1}$ is the set of recent fingerprints.

A stricter loop detector uses both exact and semantic repetition:

# $$
# R_t^{strong}

\mathbb{1}
\left[
\operatorname{count}(f_t,H_t) \geq 2
\right]
+

\mathbb{1}
\left[
\operatorname{count}_{semantic}(a_t,H_t) \geq 2
\right]

$$

### Cost penalty

Normalize cost to $$:[^0][1]

$$\tilde{C}_t = \min\left(\frac{c_t}{c_{\max}}, 1\right)$$

Then:

$$C_t = \tilde{C}_t$$

This prevents the agent from repeatedly buying tiny amounts of information at high marginal cost.

### Rolling loop score

Do not stop after one low-gain action. Maintain an exponentially weighted moving average:

# $$
# \bar{G}_t

\beta G_t
+
(1-\beta)\bar{G}_{t-1}

$$

where:

$$G_t = APG_t$$

and $0 < \beta \leq 1$. A common choice is $\beta = 0.3$.

Also maintain a finite-window sum:

# $$
# G_t^{(n)}

\sum_{i=t-n+1}^{t} G_i

$$

Define the loop indicator:

$$L_t = \mathbb{1}\left[G_t^{(n)} \le \theta_L\right]$$

with typical defaults:

$$n = 5, \qquad \theta_L = 0$$

So if the last five actions produce zero or negative net progress, the controller declares a loop.

### Action-value function

Before executing an action, estimate its expected value:

$$
Q(s_t,a_t)
$$

\mathbb{E}
\left[
G_{t+1}
\mid s_t,a_t
\right]

\lambda_c c(a_t)

\lambda_r r(a_t)

$$

where:

$$
r(a_t)
$$

estimated risk of failure or side effect

$$

The agent should choose:

$$
a_t^\star
$$

\arg\max_{a \in \mathcal{A}_{allowed}}
Q(s_t,a)

$$

subject to:

$$B_{t+1} \geq 0$$

and:

$$\text{Risk}(a_t) \leq \rho_{\max}$$

This turns loop prevention into a constrained decision problem, not just a post-hoc detector.

### Exploration versus exploitation

Use Thompson sampling to decide whether to try a promising but uncertain strategy.

For each strategy $a$, maintain a posterior over its success probability:

$$\theta_a \sim \text{Beta}(\alpha_a, \beta_a)$$

After an outcome $y_t \in \{0, 1\}$:

$$\alpha_a \leftarrow \alpha_a + y_t$$

$$\beta_a \leftarrow \beta_a + (1 - y_t)$$

Sample:

$$\tilde{\theta}_a \sim \text{Beta}(\alpha_a, \beta_a)$$

Then choose:

$$
a_t^\star
$$

\arg\max_a \tilde{\theta}_a

$$

Thompson sampling naturally balances exploration and exploitation by sampling from each action's posterior belief. [4] [5] [6]

For AgentBudget, use a constrained variant:

$$
a_t^\star
$$

\arg\max_{a \in \mathcal{A}_{feasible}}
\tilde{\theta}_a

$$

where feasibility requires predicted cost and risk to fit the remaining budget.

### Stop / recover policy

Define the controller action:

# $$
# \pi(L_t,G_t^{(n)},B_t)

$$
\begin{cases}
\text{continue}, & G_t^{(n)} > \theta_L \\
\text{warn}, & -\theta_W < G_t^{(n)} \le \theta_L \\
\text{replan}, & G_t^{(n)} \le -\theta_W \\
\text{compress}, & |s_t| > \theta_{\text{ctx}} \\
\text{stop}, & G_t^{(n)} \le \theta_{\text{stop}} \vee B_t \le 0 \\
\text{escalate}, & \text{unsafe failure} \vee \text{repeated unsafe action}
\end{cases}
$$

$$

A practical default:

$$
\theta_L = 0, \qquad \theta_W = 1, \qquad \theta_{\text{stop}} = -2
$$

Meaning:

- Positive rolling gain → continue
- Near-zero gain → warn and require a different approach
- Negative gain → replan
- Strongly negative gain → stop or escalate

### Duplicate-call circuit breaker

For each fingerprint $f$, maintain:

$$
n_f = \text{count}(f, H_t)
$$

Block execution when:

$$
n_f \ge k_{\max}
$$

with $k_{\max} = 2$ or $3$.

Instead of executing, return a structured observation:

# $$
# o_t^{blocked}

“Duplicate action detected. Change arguments, tool, or strategy.”

$$

This is cheaper than executing the same failing call again and gives the model an explicit recovery signal. [7] [8]

### Task success value

At the end, define the task utility:

$$
U = \mathbb{1}[\text{success}]

\lambda_{\text{cost}} \frac{C_{\text{total}}}{C_{\max}}

\lambda_{\text{time}} \frac{T_{\text{total}}}{T_{\max}}

\lambda_{\text{risk}} R_{\text{total}}
$$

The system objective across many tasks is:

$$\max_{\pi} \mathbb{E}_{\tau \sim \mathcal{D}} \left[ U(\tau, \pi) \right]$$

This is what AgentBudget should optimize: **successful task completion under cost, latency, and safety constraints**, not raw step count or token count alone.

### Benchmark metrics

Report these formulas in AgentBudgetBench:

$$
\text{Success Rate} = \frac{\#\text{successful tasks}}{\#\text{total tasks}}
$$

# $$
# Cost per Successful Task

\frac{\sum_{i \in successful} C_i}
{#successful tasks}

$$

# $$
# Wasted Action Rate

\frac{#{t:G_t \leq 0}}
{#total actions}

$$

# $$
# Loop Escape Rate

\frac{#tasks recovered after low-gain warning}
{#tasks entering low-gain state}

$$

# $$
# Budget Compliance

\frac{#tasks within budget}
{#total tasks}

$$

# $$
# Efficiency-Adjusted Success

Success Rate
\times
\left(
1-
\frac{C_{used}}{C_{\max}}
\right)

$$

That last metric resembles resource-aware agent evaluation: high accuracy at unbounded cost is not production-ready. [9] [10]

### Minimal implementation

```
import hashlib, json, math
from collections import deque

def h(obj):
    return hashlib.sha256(
        json.dumps(obj, sort_keys=True, default=str).encode()
    ).hexdigest()

def cosine(a, b):
    dot = sum(x * y for x, y in zip(a, b))
    na = math.sqrt(sum(x * x for x in a))
    nb = math.sqrt(sum(y * y for y in b))
    return dot / (na * nb + 1e-9)

class LoopController:
    def __init__(
        self,
        window=5,
        beta=0.3,
        warn=-0.5,
        replan=-1.0,
        stop=-2.0,
        cost_cap=0.10,
        duplicate_threshold=0.92,
    ):
        self.window = window
        self.beta = beta
        self.warn = warn
        self.replan = replan
        self.stop = stop
        self.cost_cap = cost_cap
        self.duplicate_threshold = duplicate_threshold

        self.gains = deque(maxlen=window)
        self.ema = 0.0
        self.fingerprints = deque(maxlen=10)
        self.outputs = deque(maxlen=10)
        self.args_embeddings = deque(maxlen=10)

    def action_fingerprint(self, tool, args):
        return h({"tool": tool, "args": args})

    def evidence_novelty(self, output):
        if not self.outputs:
            return 1.0

        # Replace with embedding cosine similarity in production.
        a = set(output.lower().split())
        max_sim = 0.0

        for prior in self.outputs:
            b = set(prior.lower().split())
            union = len(a | b) or 1
            max_sim = max(max_sim, len(a & b) / union)
```

```
        return 1 - max_sim

    def score(
        self,
        prev_state_hash,
        curr_state_hash,
        prev_milestones,
        curr_milestones,
        tool,
        args,
        output,
        cost,
        estimated_information_gain=0.0,
    ):
        gain = 0.0

        # State change
        if curr_state_hash != prev_state_hash:
            gain += 1.0

        # Milestone progress
        gain += max(0, curr_milestones - prev_milestones)

        # Information gain
        gain += 0.8 * estimated_information_gain

        # Evidence novelty
        gain += 0.8 * self.evidence_novelty(output)

        # Repetition penalty
        fp = self.action_fingerprint(tool, args)
        if fp in self.fingerprints:
            gain -= 2.0

        # Cost penalty
        gain -= 0.1 * min(cost / self.cost_cap, 1.0)

        # Update rolling statistics
        self.gains.append(gain)
        self.ema = self.beta * gain + (1 - self.beta) * self.ema
        self.fingerprints.append(fp)
        self.outputs.append(output)

        return gain

    def window_gain(self):
        return sum(self.gains)

    def decision(self):
        g = self.window_gain()

        if len(self.gains) < self.window:
            return "continue"

        if g <= self.stop:
```

```
            return "stop_or_escalate"
        if g <= self.replan:
            return "replan"
        if g <= self.warn:
            return "warn_change_approach"
        return "continue"
```

### What is genuinely new

You are not inventing entropy or Thompson sampling. Your contribution is the **Agent Progress Gain formulation:**

**$$**
**APG_t =**
**state progress**
**+**
**milestone progress**
**+**
**estimated information gain**
**+**
**evidence novelty**

**repetition**

cost

$$

combined with:

$$\pi = \text{argmax}_{a \in \mathcal{A}_{\text{feasible}}} Q(s, a)$$

and a budget-aware stopping rule.

That is a defensible, publishable, and implementable contribution: **a closed-loop controller that distinguishes productive iteration from wasteful looping using verifiable task progress, information gain, and resource constraints.** [1] [2] [4]

⁂

1. https://www.teach.cs.toronto.edu/~csc311notes/decision_trees/dt_infogain.html
2. https://www.cs.cmu.edu/~epxing/Class/10701-11f/HW/hw1.pdf
3. https://developers.google.com/machine-learning/glossary/metrics
4. https://stanford.edu/~ashlearn/RLForFinanceBook/MultiArmedBandits.pdf
5. https://www.lancaster.ac.uk/stor-i-student-sites/libby-daniells/2020/01/27/multi-armed-bandits-thompson-sampling/
6. https://en.wikipedia.org/wiki/Thompson_sampling

7. https://dev.to/adamo_software/tool-use-api-design-for-llms-5-patterns-that-prevent-agent-loops-and-silent-failures-f29
8. https://www.supra-wall.com/en/learn/ai-agent-infinite-loop-detection
9. https://ai-data-base.com/en/paper/2608-00805
10. https://arxiv.org/pdf/2608.00805.pdf
11. https://vllm-semantic-router.com/docs/v0.1/overview/semantic-router-overview/
12. https://hal.science/hal-01215254v1/preview/Viappiani_BanditsReset_ADT2013.pdf
13. https://www.sciencedirect.com/topics/computer-science/information-gain
14. https://courses.cs.washington.edu/courses/cse446/20wi/Lecture3/03_InformationGain.pdf
15. https://homes.cs.washington.edu/~shapiro/EE596/notes/InfoGain.pdf
16. https://www.lancaster.ac.uk/stor-i-student-sites/james-neill/wp-content/uploads/sites/47/2023/06/RT1_Report.pdf
17. https://people.maths.bris.ac.uk/~maajg/teaching/stochopt/thompson.pdf
18. https://en.wikipedia.org/wiki/Information_gain_(decision_tree)
19. https://ar5iv.labs.arxiv.org/html/1111.1797
20. https://perso.telecom-paristech.fr/bonald/documents/bandit.pdf